Camera Calibration

The Linear Camera Model: from 3D World Points to Pixels

Ahmed Mansour

1 Overview

A camera has two groups of parameters, and finding them is what *camera calibration* does.

1. **External parameters** (the *extrinsic matrix* M_{ext}): the position and orientation of the camera with respect to the world coordinate frame.
2. **Internal parameters** (the *intrinsic matrix* M_{int}): how the camera maps points, expressed in its own frame, onto its image plane and sensor, for example through the focal length.

The **camera model** describes the projection of 3D points to pixels in the image. We want this model to be *linear*, because that simplifies the estimation. The linear model is a single matrix, called the **projection matrix**.

Textbook treatments of this material include [1] which covers the same derivation as a lecture series. Section 9 gives the references for lens distortion and for the checkerboard method.

2 Forward Imaging Model: 3D to 2D

Figure 1 shows the three coordinate systems involved: the world frame W, the camera frame C and the image. A point $\mathbf{X}_w = (x_w, y_w, z_w)^\top$ is first transformed into the camera frame, $\mathbf{X}_c = (x_c, y_c, z_c)^\top$ (coordinate transformation, M_{ext}), and is then projected onto the image to give the pixel $\mathbf{u} = (u, v)^\top$ (perspective projection, M_{int}):

$$\mathbf{X}_w \xrightarrow[\text{coordinate transformation}]{M_{\text{ext}}} \mathbf{X}_c \xrightarrow[\text{perspective projection}]{M_{\text{int}}} \mathbf{u}.$$

By similar triangles (focal length f),

$$\frac{x_i}{f} = \frac{x_c}{z_c}, \qquad \frac{y_i}{f} = \frac{y_c}{z_c} \qquad \Longrightarrow \qquad x_i = f\,\frac{x_c}{z_c}, \quad y_i = f\,\frac{y_c}{z_c}.$$

Note on the sign convention. In a real pinhole camera the image forms *behind* the pinhole and is upside down, which would put a minus sign in front of f. Placing the (virtual) image plane in front of the pinhole at $z_c = f$, as in Figure 1, removes the flip and gives the formulas above. The z_c axis points toward the scene, so visible points have $z_c > 0$.

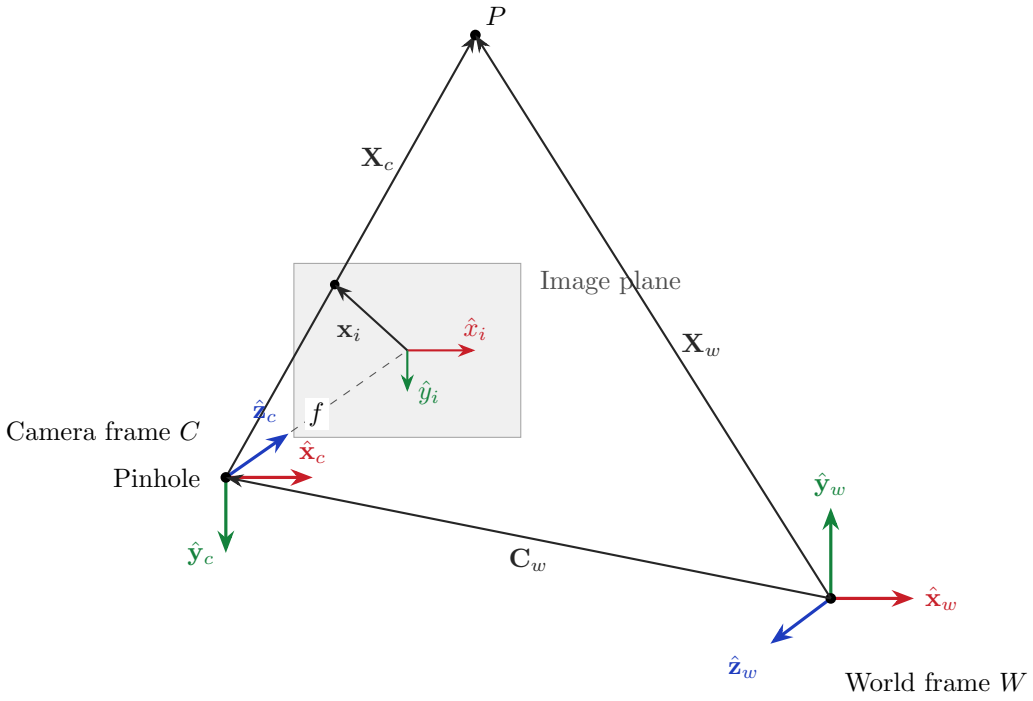


Figure 1: Forward imaging model. The image plane is drawn *in front of* the pinhole, at distance f along the optical axis $\hat{\mathbf{z}}_c$. $\mathbf{C}_w$ is the camera position in the world frame; $\mathbf{X}_w$ and $\mathbf{X}_c$ are the scene point P in world and camera coordinates; $\mathbf{x}_i$ is its image on the image plane.

3 Image Plane to Image Sensor Mapping

The image plane is measured in *millimetres*, but the sensor is measured in *pixels* (Figure 2). Since pixels might be rectangular, we assume pixel densities m_x and m_y (pixels/mm) along x and y:

$$u = m_x\, x_i = m_x f\, \frac{x_c}{z_c} = f_x\, \frac{x_c}{z_c},$$
$$v = m_y\, y_i = m_y f\, \frac{y_c}{z_c} = f_y\, \frac{y_c}{z_c},$$

where $f_x = m_x f$ and $f_y = m_y f$ are the focal length expressed in pixels.

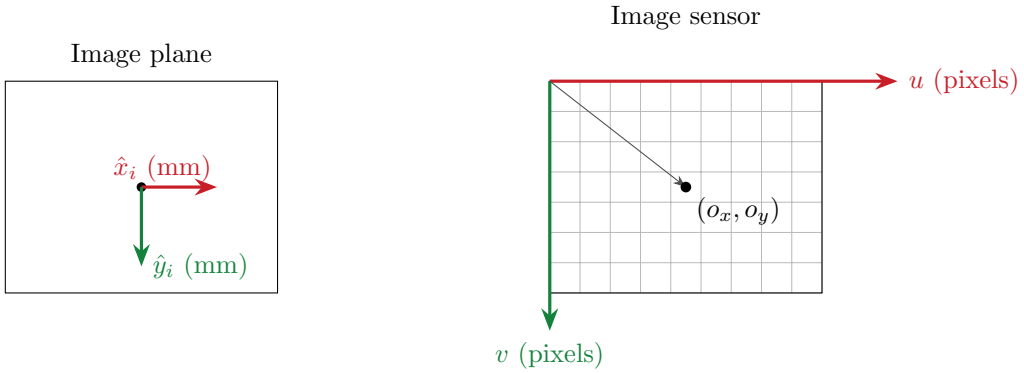


Figure 2: Image plane (millimetres, origin at the principal point) and image sensor (pixels, origin at the top-left corner). The grey arrow points to the principal point (o_x, o_y).

The top-left corner of the image sensor is treated as the origin. The point where the optical axis

pierces the sensor is the **principal point** (o_x, o_y), so

$$u = f_x \frac{x_c}{z_c} + o_x, \qquad v = f_y \frac{y_c}{z_c} + o_y. \tag{1}$$

4 Perspective Projection

In Equation (1), (f_x, f_y, o_x, o_y) are the **intrinsic parameters** of the camera. They represent the internal geometry of the camera.

The equations are not linear in (x_c, y_c, z_c) because of the division by z_c. To linearise them, we use **homogeneous coordinates**.

5 Homogeneous Coordinates

The homogeneous representation of a 2D point $\mathbf{u} = (u, v)$ is a 3D point $\tilde{\mathbf{u}} = (\tilde{u}, \tilde{v}, \tilde{w})$. The third coordinate $\tilde{w} \neq 0$ is fictitious, and

$$u = \frac{\tilde{u}}{\tilde{w}}, \qquad v = \frac{\tilde{v}}{\tilde{w}},$$

$$\begin{bmatrix} u \\ v \\ 1 \end{bmatrix} \simeq \begin{bmatrix} \tilde{w}\, u \\ \tilde{w}\, v \\ \tilde{w} \end{bmatrix} = \begin{bmatrix} \tilde{u} \\ \tilde{v} \\ \tilde{w} \end{bmatrix} = \tilde{\mathbf{u}},$$

where $\simeq$ means "equal up to a non-zero scale factor". Every point on the line L through the origin (except the origin itself) represents the same 2D point $\mathbf{u} = (u, v)$ (Figure 3).

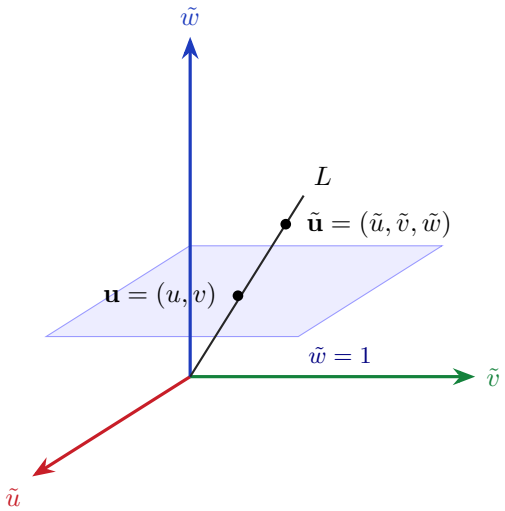


Figure 3: The 2D point $\mathbf{u} = (u, v)$ lies on the plane $\tilde{w} = 1$. Every point of the line L through the origin and $\mathbf{u}$ is a homogeneous representation $\tilde{\mathbf{u}}$ of it.

With the same principle in 3D: $\mathbf{X} = (x, y, z) \Rightarrow \tilde{\mathbf{X}} = (\tilde{x}, \tilde{y}, \tilde{z}, \tilde{w})$ with $\tilde{w} \neq 0$, and

$$\begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix} \simeq \begin{bmatrix} \tilde{w}\, x \\ \tilde{w}\, y \\ \tilde{w}\, z \\ \tilde{w} \end{bmatrix} = \begin{bmatrix} \tilde{x} \\ \tilde{y} \\ \tilde{z} \\ \tilde{w} \end{bmatrix} = \tilde{\mathbf{X}}.$$

6 The Intrinsic Matrix: Linear Form

Remember

$$u = f_x \frac{x_c}{z_c} + o_x, \qquad v = f_y \frac{y_c}{z_c} + o_y.$$

Multiplying through by z_c (a scale factor, which is allowed in homogeneous coordinates) gives

$$\begin{bmatrix} u \\ v \\ 1 \end{bmatrix} \simeq \begin{bmatrix} u\, z_c \\ v\, z_c \\ z_c \end{bmatrix} = \begin{bmatrix} f_x\, x_c + o_x\, z_c \\ f_y\, y_c + o_y\, z_c \\ z_c \end{bmatrix} = \begin{bmatrix} f_x & 0 & o_x & 0 \\ 0 & f_y & o_y & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} x_c \\ y_c \\ z_c \\ 1 \end{bmatrix}.$$

The **calibration matrix** K and the **intrinsic matrix** M_{int} are

$$K = \begin{bmatrix} f_x & 0 & o_x \\ 0 & f_y & o_y \\ 0 & 0 & 1 \end{bmatrix}, \qquad M_{\text{int}} = \begin{bmatrix} f_x & 0 & o_x & 0 \\ 0 & f_y & o_y & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} = \begin{bmatrix} K \mid \mathbf{0} \end{bmatrix}, \qquad \tilde{\mathbf{u}} = M_{\text{int}}\, \tilde{\mathbf{X}}_c.$$

7 Extrinsic Parameters

The extrinsic parameters are the position $\mathbf{C}_w$ and the orientation R of the camera in the world coordinate frame W.

$$R = \begin{bmatrix} r_{11} & r_{12} & r_{13} \\ r_{21} & r_{22} & r_{23} \\ r_{31} & r_{32} & r_{33} \end{bmatrix} \begin{matrix} \leftarrow \text{ direction of } \hat{\mathbf{x}}_c \text{ in the world frame} \\ \leftarrow \text{ direction of } \hat{\mathbf{y}}_c \text{ in the world frame} \\ \leftarrow \text{ direction of } \hat{\mathbf{z}}_c \text{ in the world frame} \end{matrix}$$

Note: R is orthonormal ($R^\top R = I$, and for a proper rotation $\det R = 1$). The transformation from world to camera coordinates is

$$\begin{bmatrix} x_c \\ y_c \\ z_c \end{bmatrix} = \begin{bmatrix} r_{11} & r_{12} & r_{13} \\ r_{21} & r_{22} & r_{23} \\ r_{31} & r_{32} & r_{33} \end{bmatrix} \begin{bmatrix} x_w \\ y_w \\ z_w \end{bmatrix} + \begin{bmatrix} t_x \\ t_y \\ t_z \end{bmatrix}, \qquad \text{i.e.} \qquad \mathbf{X}_c = R\, \mathbf{X}_w + \mathbf{t}.$$

After linearisation (homogeneous coordinates):

$$M_{\text{ext}} = \begin{bmatrix} R_{3\times 3} & \mathbf{t} \\ \mathbf{0}_{1\times 3} & 1 \end{bmatrix} = \begin{bmatrix} r_{11} & r_{12} & r_{13} & t_x \\ r_{21} & r_{22} & r_{23} & t_y \\ r_{31} & r_{32} & r_{33} & t_z \\ 0 & 0 & 0 & 1 \end{bmatrix}, \qquad \tilde{\mathbf{X}}_c = M_{\text{ext}}\, \tilde{\mathbf{X}}_w.$$

How t relates to the camera position. The camera centre $\mathbf{C}_w$ is the point that maps to the origin of the camera frame: $\mathbf{0} = R\,\mathbf{C}_w + \mathbf{t}$, so $\mathbf{t} = -R\,\mathbf{C}_w$. The translation $\mathbf{t}$ is the *world origin as seen from the camera*, not the camera position itself. Substituting $\mathbf{t} = -R\,\mathbf{C}_w$ gives $P = KR\,[\,I \mid -\mathbf{C}_w\,]$, the form used in [**?**, §6.1].

8 The Projection Matrix P

Camera to pixel: $\tilde{\mathbf{u}} = M_{\text{int}}\, \tilde{\mathbf{X}}_c$. World to camera: $\tilde{\mathbf{X}}_c = M_{\text{ext}}\, \tilde{\mathbf{X}}_w$. Combining them,

$$\tilde{\mathbf{u}} = M_{\text{int}}\, M_{\text{ext}}\, \tilde{\mathbf{X}}_w = P\, \tilde{\mathbf{X}}_w, \qquad \begin{bmatrix} \tilde{u} \\ \tilde{v} \\ \tilde{w} \end{bmatrix} = \begin{bmatrix} p_{11} & p_{12} & p_{13} & p_{14} \\ p_{21} & p_{22} & p_{23} & p_{24} \\ p_{31} & p_{32} & p_{33} & p_{34} \end{bmatrix} \begin{bmatrix} x_w \\ y_w \\ z_w \\ 1 \end{bmatrix},$$

so P is a 3×4 matrix, and with $M_{\text{int}} = [K \mid \mathbf{0}]$ and M_{ext} as above,

$$\boxed{P = M_{\text{int}}\, M_{\text{ext}} = K\,[\,R \mid \mathbf{t}\,]}$$

What a single image cannot tell us: depth

From the previous equations, a pixel (u, v) determines the *ray* on which the scene point lies, but never its *depth*. Inverting Equation (1) gives the point as a function of the unknown depth z_c:

$$x_c = \frac{z_c}{f_x}\,(u - o_x), \qquad y_c = \frac{z_c}{f_y}\,(v - o_y), \qquad z_c > 0.$$

Every value of $z_c > 0$ gives a different point on the same ray, which is why depth needs extra information, for example a second camera.

9 Beyond the Linear Model

- The linear model above ignores **lens distortion**. A real calibration also estimates distortion coefficients (radial k_1, k_2, k_3 and tangential p_1, p_2) that describe how the normalised image coordinates x_c/z_c and y_c/z_c deviate from the ideal pinhole model before K maps them to pixels. The exact conventions used by OpenCV are in [2].
- Calibrating with a checkerboard (OpenCV `calibrateCamera`) estimates K (that is, f_x, f_y, o_x and o_y) and the distortion coefficients D. For each image of the board it also gives the pose of the board relative to the camera, which is an M_{ext} with the board as the world frame.
- For a stereo pair, `stereoCalibrate` estimates the rotation R and translation $\mathbf{t}$ between the two cameras, which is an extrinsic transformation from the left camera frame to the right one. After rectification the depth follows from the disparity d as $Z = f\,B/d$, with baseline B.

Companion code: https://github.com/AMansour5/CameraCalibrator

References

[1] S. K. Nayar, *First Principles of Computer Vision* (lecture series, including the lecture "Camera Calibration"), Columbia University. https://fpcv.cs.columbia.edu; lecture video: https://www.youtube.com/watch?v=GUbWsXU1mac.

[2] OpenCV documentation, module *calib3d*: "Camera Calibration and 3D Reconstruction" (version 4.13.0). https://docs.opencv.org/4.13.0/d9/d0c/group__calib3d.html.